IN 2023 · IN THE US ALONE

nearly 3.8 million

adult hospital readmissions within 30 days

a 13.5% readmission rate for adult hospital stays

Source: AHRQ HCUP, Nationwide Readmissions Database, 2023 (Table A6): 3,769,100 readmissions
hcup-us.ahrq.gov/db/nation/nrd/NRDIntroduction2023.pdf

THE SAFETY GATE

Four unclear instructions. Four tested rules.

AI extracts the letter. Fixed Python rules flag ambiguity instead of inventing missing medical instructions. Nothing unclear is treated as fact.

01 - NO END DATE

Apixaban: “Continue as directed.”

⚠ Keep the medicine; flag the end date as unspecified.

⚠ No end date — test_data/letter1_margaret_knee_replacement_SYNTHETIC_v4.pdf

02 - UNREADABLE DOSE

Dose: “2 [?] tablets”

⚠ Flag for a person to confirm. Do not infer a dose.

⚠ Unreadable — tests/ingestion/test_validator.py

03 - CONFLICTING DATES

Review: 10 October / 12 October

⚠ Preserve both dates; do not choose one.

⚠ Two dates — tests/ingestion/test_extraction_scenarios.py

04 - MISSING DOSE

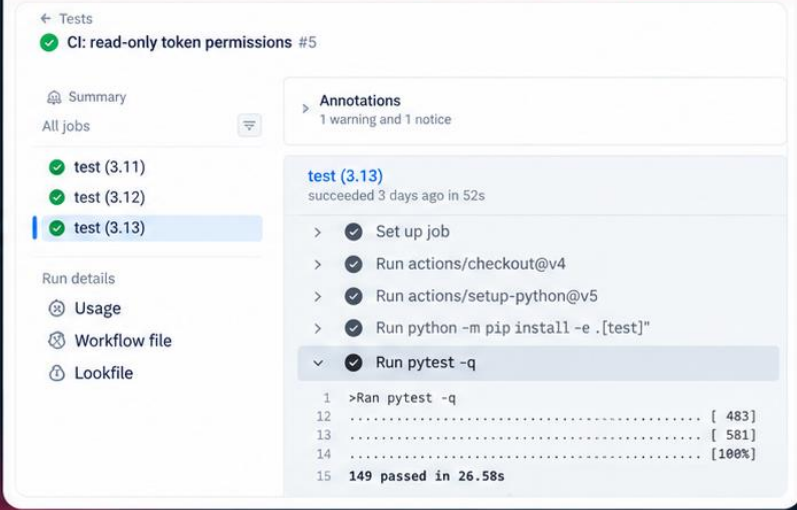
“Amoxicillin 500 mg”

⚠ Leave dose blank; never invent the amount.

⚠ No dose — tests/ingestion/test_phase4.py

LIVE TEST EVIDENCE · GITHUB ACTIONS (discharge-letter-check package repo)

✓ 149 tests passed



Open source: discharge-letter-check • MIT • v0.1.0 • 149 tests — View on GitHub

BUILT ON AWS · EU-WEST-2

How AfterWard works

From a discharge letter to a safe, personalized recovery plan — with proactive reminders and voice experience.

Green tags = hackathon requirement met

1 A letter becomes a plan

A discharge letter is turned into a safe, personalized plan.

Letter upload
Patient or hospital uploads discharge letter (about 15 seconds)

↓

Amazon S3
Private, encrypted bucket

↓

Amazon SQS
Queue, with a dead-letter queue and alarm

↓

AWS Lambda
Reads the letter's text, then Claude on Amazon Bedrock fills in a fixed form

↓

Safety gate (open source) ✓ OPEN SOURCE
discharge-letter-check (MIT)
Flags unclear items for a person to confirm

2 She talks to her plan

She asks about her plan using voice or screen, and AfterWard uses safe tools to help.

Simulated Alexa+ client ✓ SIMULATED ALEXA+
Next.js · voice in, voice out
plus an Alexa skill in Amazon's simulator

↓

Agent
Crisis check first: a fixed safe reply, no AI
Then Claude on Amazon Bedrock picks a tool

↓

AfterWard MCP server ✓ SELF-HOSTED MCP
Amazon ECS on Fargate, behind ALB + CloudFront
Streamable HTTP · spec 2025-11-25

↓

Nine tools, one patient
6 read: today, week, a day, medicines, plan, her letter
3 write: task done, medicine taken, undo
2 prompts · **MCP App**

3 AfterWard speaks first

Proactive reminders keep her on track, and she can always reply.

Amazon EventBridge Scheduler
Runs the reminder check every 5 minutes

↓

AWS Lambda
Finds what is due in the next 30 minutes

↓

Pending reminder
The client speaks it out loud, unasked.
Reminder text is built by code, not the model.

↓

She answers
"I'm back from my walk"
Ticks the walk off, saved in her event history

Data, kept per patient
Each patient's data stays separate and private, in the UK (eu-west-2).

Amazon DynamoDB
Recovery plan · event history

Amazon S3 Vectors
Her own letter · Titan Text Embeddings V2

✓ **AWS BUILDER**
14 AWS services, each documented

AfterWard is a proactive voice companion. Amazon EventBridge starts each reminder before it is due, so she never has to ask.

ALEXA+ TRACK · AWS BUILDER · OPEN SOURCE · EU-WEST-2

How AfterWard meets the brief

Every claim links to a file in the repo.

Green tags = hackathon requirement met

1 Patient & input channels

Margaret
talks or types

Simulated Alexa+
Next.js web app on screen

Mobile app PLANNED
Reminders, tasks and plan view

Hospital systems PLANNED
Discharge summary (PDF / HL7 / FHIR)

2 Edge & identity

Amazon CloudFront PLANNED
Security, DDoS, rate limits

Amazon Cognito PLANNED
User login, roles (patients, carers, clinicians)

Amazon API Gateway PLANNED
REST/HTTP endpoints for web, mobile, MCP

3 AfterWard MCP server (core services on ECS Fargate) R1

AfterWard MCP server
Python MCP SDK · 9 tools
ECS Fargate behind ALB + CloudFront

- get_today_tasks
- get_medications
- report_medication_taken
- get_recovery_plan
- get_week_summary
- search_discharge_instructions
- report_task_completed
- next_up
- get_patient_context

Safety gate (open source) R7
discharge-letter-check (MIT)
Validates source text, flags missing or unclear items

4 Data layer

Amazon S3 Vectors R3
Her letter, filtered by patient (Titan Text Embeddings V2)

DynamoDB: event history R3
Append only log of what she did

DynamoDB: recovery state R5
Plan, today's tasks, next up and extracted documents

Amazon S3 R5
Discharge letters (PDF) and extracted documents

Amazon SQS R4
Task and reminder queue (fan-out to workers)

Amazon EventBridge R4
Schedules, daily summaries, follow-ups

5 AI / ML services

Amazon Bedrock R6
LLM for grounded answers (using RAG + letter context)

S3 Vectors PLANNED
search the patient's own letter

Reminder Lambda R4
Every 5 minutes, saves a check-in for AfterWard to speak

Bedrock extraction + discharge-letter-check R5
Extract text and structure from discharge letters

Hackathon requirements

Status today, with the file that proves it

R1	**Self-hosted MCP server, spec 2025-11-25** Streamable HTTP, on ECS Fargate app/mcp/server.py	MET
R2	**MCP server really called in code** Every read and write goes through it app/mcp/client.py	MET
R3	**Simulated Alexa+ experience, with source** Labelled web app + Alexa skill in app/skills/	MET
R4	**Public repo with an open-source licence** MIT licence in LICENSE + repository is public	MET
R5	**Clear setup and run instructions** README for judges, checked by a fresh-clone test	MET
R6	**AWS Builder: services with documented use** 14 AWS services, each linked to its file - docs/aws-integrations.md	MET
R7	**Open Source: a new, separate project** discharge-letter-check · MIT · v0.1.0 149 tests - used by AfterWard	MET

Tested: 1,269 Python · 138 web

Tech choices and the trade-offs we accepted

Two models, one switch
Sonnet 4.5 (final runs)
Haiku 4.5 (testing)
Best quality with faster for testing.

Rules gate, not an AI checker
Open-source Python package
Deterministic, testable and safe.

S3 Vectors, not OpenSearch
Simple and reliable for our scale.
Swappable if needed.

DynamoDB, not SQL
Fast patient state and task history.
Designed for high concurrency.

MCP on Fargate, not Lambda
Multiple tools, state and scheduled jobs.
Easier to maintain.

Browser voice, not Polly
Alexa+ voice when access opens.
Polly is not used for testing or demos.

AFTERWARD: A LIVING RECOVERY PLAN FOR ALEXA+

Thank you

to the Alexa team and Amazon Developer.

For a wonderful opportunity to learn and build a unique experience at Build, Ship, Shape.
We'd also love the chance to build what comes next because, after all,
no one should have to remember their recovery alone.



Real people
Inspired by patients like Margaret, and everyone recovering at home.

Responsible AI
Grounded in their own medical information, with safety built in.

Bigger impact
A step towards more supported recoveries, aims to support fewer readmissions and healthier lives at home.

AfterWard (main app)
github.com/charvimaurya/post-discharge-mcp

Open source: discharge-letter-check
github.com/charvimaurya/discharge-letter-check

Team: Shilpa Agarwal and Charvi Maurya

A living plan: reminds, records progress, and flags uncertainty for a person.

Discharge is one moment. Recovery happens every day.

WHAT IS DONE, AND WHAT COMES NEXT

The MVP

A working end-to-end demo that turns a discharge letter into a safe, personalized recovery plan — with a clear path to what is next.

DONE

Built and running

A complete, working demo that shows the core value and safety approach.

- **Upload a discharge letter**
  It becomes a plan she can talk to.
- **Answers from her own letter**
  With the page they came from.
- **Safety gate (open source package)**
  Flags unclear items for a person to confirm.
- **Safe replies before any AI runs**
  A crisis check that replies safely.
- **Nine MCP tools, two prompts and an app**
  On spec 2025-11-25.
- **Proactive voice reminders**
  Reminder check runs on a schedule; the demo runs it locally.
- **Alexa skill and web voice client**
  With a text fallback.

SIMULATED

Simulated or synthetic

A realistic demo using simulated patients, data and Alexa+.

- **Alexa+ in a web app**
  The Alexa skill runs in Amazon's simulator.
- **Every discharge letter is synthetic**
  No real patient data.
- **One shared demo patient**
  Margaret, with a full recovery plan.
- **Fixed demo date**
  The demo clock is set to 23 October for the recording.

NEXT PHASE

What comes next

From a demo to a pilot and global rollout.

- **Secure sign-in and patient consent**
- **Reminder times she chooses**
  Even when her letter gives none (the letter stays in charge).
- **Unclear instructions to her care team**
- **Daily progress reports**
  For family and care team.
- **One-step calling to emergency services**
  Or her carer.
- **Photo letters, and voice in her own language**
- **Integration with hospital systems**
- **Clinical pilot before any real patient**

AfterWard is a proactive voice companion. Amazon EventBridge starts each reminder before it is due, so she never has to ask.

WHAT'S NEXT · TARGET ARCHITECTURE · DATA STAYS IN THE EU

AfterWard at scale: built for millions of patients

Same MCP tools and safety gate as today. Designed for scale, security and reusability.

EU-WEST-2 — Data stays in the EU

Open source — MIT licensed

Scalable — Millions of patients

1 Patient & input channels

- **Margaret** — talks or types
- **Simulated Alexa+** — Next.js web app on screen
- **Mobile app** PLANNED — Reminders, tasks and plan view
- **Hospital systems** PLANNED — Discharge summary (PDF / HL7 / FHIR)

2 Edge & identity

- **Amazon CloudFront +** — Security, rate limits PLANNED
- **Amazon Cognito** PLANNED — User login, roles (patients, carers, clinicians)
- **Amazon API Gateway** PLANNED — REST/HTTP endpoints for web, mobile, MCP

3 AfterWard MCP server (core services on ECS Fargate) R1

AfterWard MCP server
Python MCP SDK · 9 tools
ECS Fargate behind ALB + CloudFront

- get_today_tasks
- get_medications
- report_medication_taken
- get_recovery_plan
- get_week_summary
- search_discharge_instructions
- report_task_completed
- next_up
- get_patient_context

Safety gate (open source) R7
discharge-letter-check (MIT)
Validates source text, flags missing or unclear items

4 Data layer

- **Amazon S3 Vectors** R3 — Her letter, filtered by patient (Titan Text Embeddings V2)
- **DynamoDB: event history** R3 — Append only log of what she did
- **DynamoDB: recovery state** R5 — Plan, today's tasks, next up and extracted documents
- **Amazon S3** R5 — Discharge letters (PDF) and extracted documents
- **Amazon SQS** R4 — Task and reminder queue (fan-out to workers)
- **Amazon EventBridge** R4 — Schedules, daily summaries, follow-ups

5 AI / ML services

- **Amazon Bedrock** R6 — LLM for grounded answers (using RAG + letter context)
- **Knowledge base** PLANNED — Guidelines, discharge instructions and policies
- **Reminder Lambda** R4 — Every 5 minutes, saves a check-in for AfterWard to speak
- **Bedrock extraction + discharge-letter-check** — Extract text and structure from discharge letters

Operations & security

- **AWS IAM, encryption at rest** — Encryption at rest, least privilege access
- **Amazon CloudWatch** — Logs, metrics, alarms, cost monitoring
- **CI/CD (GitHub Actions)** PLANNED — Tests, build, deploy (ECS Fargate, Lambda)
- **Observability** PLANNED — Tracing, error tracking, usage analytics
- **Data governance** — EU data residency, access controls, audit logs

Tech choices and the trade-offs we accepted

Two models, one switch
Sonnet 4.5 (final runs)
Haiku 4.5 (testing)
Best quality for final runs; faster for testing.

Rules gate, not an AI checker
Open-source Python package
Deterministic, testable and safe.

S3 Vectors, not OpenSearch
Simple and reliable for our scale.
Swappable if needed.

DynamoDB, not SQL
Fast patient state and task history.
Designed for high concurrency.

MCP on Fargate, not Lambda
Multiple tools, state and scheduled jobs.
Easier to maintain.

Browser voice, not Polly
Alexa+ voice when access opens.
Polly is not used for testing or demos.

Hackathon requirements — status

R1	Self-hosted MCP server, spec 2025-11-25	MET
R2	9 MCP tools implemented	MET
R3	Simulated Alexa+ client (web)	MET
R4	Public repo with open-source licence (MIT)	MET
R5	Clear setup and run instructions (README)	MET
R6	AWS Builder with documented services	MET
R7	Separate validation package (discharge-letter-check)	MET

Tested: 1,269 Python · 138 web

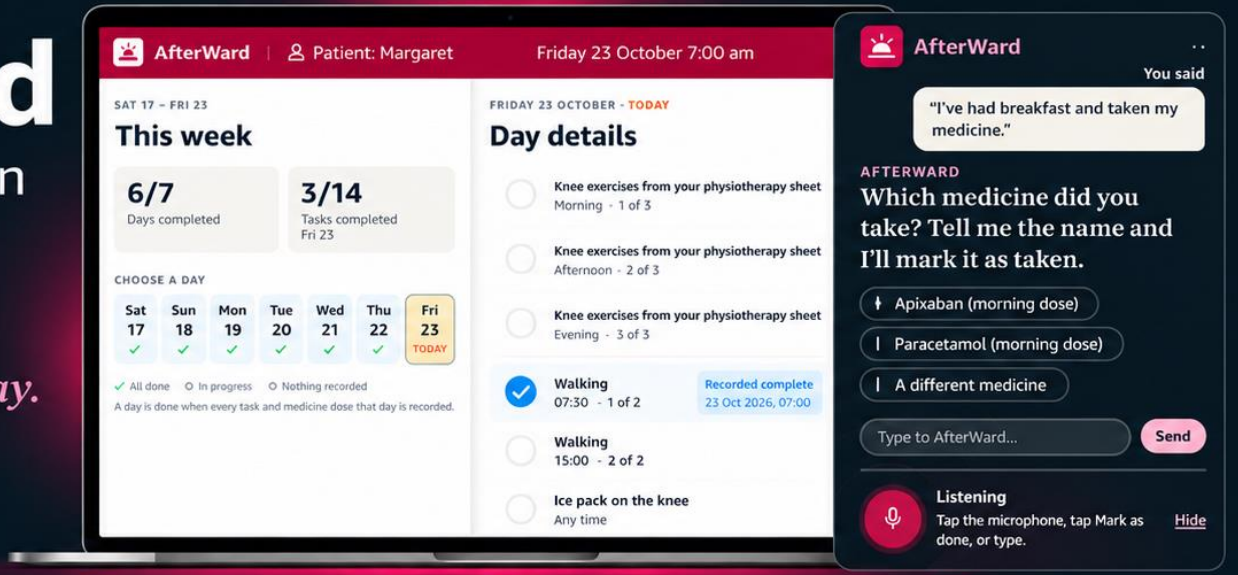
BUILD, SHIP, SHAPE: AMAZON DEVELOPER HACKATHON

AfterWard

A Living Recovery Plan for Alexa+

Discharge is one moment. Recovery happens every day.

AfterWard turns a discharge letter into a personalized, day-by-day plan that updates as you recover.



Alexa skill
Classic skill, and a simulated Alexa+ web app

MCP
9 tools for a living recovery plan

AWS
Secure, scalable cloud architecture

Open Source
Reusable validation package